

Source Code Project MCBKD

File này gom các file source/config chính để in hoặc xuất PDF. Đã loại bỏ thư mục build, thư viện ngoài, ảnh và tài liệu phụ.

Lưu ý: `src/project_config.h` đang có WiFi password và ThingsBoard token. Nếu không/chia sẻ công khai, nên che hoặc đổi các giá trị này.

Mục lục

- `platformio.ini`
- `partitions_gateway_ota.csv`
- `src/project_config.h`
- `src/packet_protocol.h`
- `src/packet_protocol.cpp`
- `src/dht22_sensor.h`
- `src/dht22_sensor.cpp`
- `src/soil_moisture.h`
- `src/soil_moisture.cpp`
- `src/node_main.cpp`
- `src/gateway_state.h`
- `src/gateway_state.cpp`
- `src/gateway_logic.h`
- `src/gateway_logic.cpp`
- `src/gateway_lora.h`
- `src/gateway_lora.cpp`
- `src/gateway_cloud.h`
- `src/gateway_cloud.cpp`
- `src/gateway_web.h`
- `src/gateway_web.cpp`
- `src/gateway_main.cpp`

platformio.ini

```
[platformio]
default_envs = gateway

[env]
platform = espressif32
board = esp32doit-devkit-v1
framework = arduino
monitor_speed = 115200
upload_speed = 921600
build_flags =
    -D CORE_DEBUG_LEVEL=1
lib_deps =
```

sandeepmistry/LoRa

```
; =====
; NODE: sensors + LoRa telemetry + ACK/command + adaptive sleep
; =====
[env:node]
build_src_filter =
    -<*>
    +<node_main.cpp>
    +<dht22_sensor.cpp>
    +<soil_moisture.cpp>
    +<packet_protocol.cpp>

; =====
; GATEWAY: LoRa RX/TX + local dashboard/export + ThingsBoard HTTP
; =====
[env:gateway]
board_build.partitions = partitions_gateway_ota.csv
build_src_filter =
    -<*>
    +<gateway_main.cpp>
    +<gateway_state.cpp>
    +<gateway_logic.cpp>
    +<gateway_cloud.cpp>
    +<gateway_lora.cpp>
    +<gateway_web.cpp>
    +<packet_protocol.cpp>
```

partitions_gateway_ota.csv

#	Name,	Type,	SubType,	Offset,	Size,	Flags
nvs,	data,	nvs,	0x9000,	0x5000,		
otadata,	data,	ota,	0xe000,	0x2000,		
app0,	app,	ota_0,	0x10000,	0x140000,		
app1,	app,	ota_1,	0x150000,	0x140000,		
spiffs,	data,	spiffs,	0x290000,	0x170000,		

src/project_config.h

```
#pragma once

#include <Arduino.h>

namespace Config
{
    constexpr uint32_t SerialBaud = 115200;
    constexpr char GatewayFirmwareVersion[] = "GW_OTA_TEST_1";
}
```

```
constexpr uint8_t NodeId = 1;
constexpr uint8_t GatewayId = 1;

constexpr uint8_t DhtPin = 27;
constexpr uint8_t SoilAdcPin = 34;
constexpr uint8_t BatteryAdcPin = 35;
constexpr uint8_t SensorPowerPin = 32;
constexpr float BatteryDividerRatio = 3.2f; // Rtop=220k, Rbottom=100k.
constexpr uint8_t PumpPin = 25;
constexpr bool PumpActiveHigh = true;
constexpr uint8_t MaxPumpSeconds = 15;
constexpr bool EnablePumpHardware = false;

constexpr float TempOffsetC = 0.0f;
constexpr float HumidityOffsetRh = 0.0f;
constexpr int SoilAdcDry = 3400;
constexpr int SoilAdcWet = 1200;
constexpr int SoilAdcMin = 100;
constexpr int SoilAdcMax = 4090;

constexpr uint8_t LoraSs = 5;
constexpr uint8_t LoraRst = 14;
constexpr uint8_t LoraDio0 = 26;
constexpr uint8_t LoraSck = 18;
constexpr uint8_t LoraMiso = 19;
constexpr uint8_t LoraMosi = 23;
constexpr long LoraFrequency = 433E6;
constexpr uint8_t LoraSpreadingFactor = 7;
constexpr long LoraSignalBandwidth = 125E3;
constexpr uint8_t LoraCodingRate = 5;
constexpr uint8_t LoraTxPowerDbm = 17;

constexpr uint16_t AckTimeoutMs = 2500;
constexpr uint16_t CommandDedupWindow = 32;

constexpr bool EnableDeepSleep = true;
constexpr uint32_t DefaultSleepMinutes = 30;

constexpr char WifiSsid[] = "Khoa";
constexpr char WifiPassword[] = "12112004";
constexpr char ThingsBoardHost[] = "eu.thingsboard.cloud";
constexpr char ThingsBoardToken[] = "QYtNEEPckyFjPteWRQ9o";
constexpr uint16_t ThingsBoardHttpPort = 80;
constexpr bool EnableCloudUpload = true;

constexpr bool EnableGatewayHttpDebug = false;
constexpr bool EnableGatewayHeartbeatLog = false;
}
```

```
#pragma once

#include <Arduino.h>

enum class CommandType : uint8_t
{
    None = 0,
    SetSleepDuration = 1,
    SetThreshold = 2,
    StartOta = 3,
    SleepNow = 4,
    StartPump = 5,
    ResendChunk = 6,
    SetFilterMode = 7,
    SetPumpTime = 8,
    SetControlMode = 9,
    SetDutyCycle = 10
};

enum class PacketKind : uint8_t
{
    Telemetry = 1,
    Ack = 2,
    Command = 3,
    OtaChunk = 4
};

struct TelemetryPacket
{
    uint8_t nodeId = 0;
    uint32_t packetId = 0;
    float temperatureC = NAN;
    float humidityRh = NAN;
    float soilMoistureVol = NAN;
    float batteryV = NAN;
    int adcFiltered = 0;
    uint8_t errorFlag = 0;
    String soilStatus;
    uint32_t configSleepMinutes = 0;
    int configSoilThresholdVol = 0;
    int configFilterMode = 0;
    int configPumpSeconds = 0;
    int configControlMode = 0;
    int configDutyCycleMode = 0;
};

struct AckPacket
{
    uint8_t nodeId = 0;
    uint32_t packetId = 0;
    bool ok = false;
    CommandType command = CommandType::None;
```

```

    int parameter = 0;
    String status;
};

struct OtaChunkPacket
{
    uint8_t nodeId = 0;
    uint32_t otaId = 0;
    uint16_t chunkIndex = 0;
    uint16_t totalChunks = 0;
    String payloadData;
    uint16_t dataCrc = 0;
};

struct OtaStatusPacket
{
    uint8_t nodeId = 0;
    uint32_t otaId = 0;
    uint16_t chunkIndex = 0;
    bool ok = false;
    String status;
};

uint16_t crc16Ccitt(const uint8_t *data, size_t length);
uint16_t crc16Ccitt(const String &text);

String commandToText(CommandType command);
CommandType commandFromText(const String &text);

String encodeTelemetry(const TelemetryPacket &packet);
String encodeAck(const AckPacket &packet);
String encodeOtaChunk(const OtaChunkPacket &packet);
String encodeOtaStatus(const OtaStatusPacket &packet);
bool decodeTelemetry(const String &line, TelemetryPacket &packet);
bool decodeAck(const String &line, AckPacket &packet);
bool decodeOtaChunk(const String &line, OtaChunkPacket &packet);
bool decodeOtaStatus(const String &line, OtaStatusPacket &packet);
bool isValidCrc(const String &line);
String getField(const String &line, const String &key);

```

src/packet_protocol.cpp

```

#include "packet_protocol.h"

static String withoutCrc(const String &line)
{
    const int crcIndex = line.lastIndexOf(",CRC=");
    if (crcIndex < 0)
    {
        return line;
    }
}

```

```
    }
    return line.substring(0, crcIndex);
}

uint16_t crc16Citt(const uint8_t *data, size_t length)
{
    uint16_t crc = 0xFFFF;
    for (size_t i = 0; i < length; i++)
    {
        crc ^= uint16_t(data[i]) << 8;
        for (uint8_t bit = 0; bit < 8; bit++)
        {
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : crc << 1;
        }
    }
    return crc;
}

uint16_t crc16Citt(const String &text)
{
    return crc16Citt(reinterpret_cast<const uint8_t *>(text.c_str()),
text.length());
}

String commandToText(CommandType command)
{
    switch (command)
    {
        case CommandType::SetSleepDuration:
            return "SET_SLEEP_DURATION";
        case CommandType::SetThreshold:
            return "SET_THRESHOLD";
        case CommandType::StartOta:
            return "START_OTA";
        case CommandType::SleepNow:
            return "SLEEP_NOW";
        case CommandType::StartPump:
            return "START_PUMP";
        case CommandType::ResendChunk:
            return "RESEND_CHUNK";
        case CommandType::SetFilterMode:
            return "SET_FILTER_MODE";
        case CommandType::SetPumpTime:
            return "SET_PUMP_TIME";
        case CommandType::SetControlMode:
            return "SET_CONTROL_MODE";
        case CommandType::SetDutyCycle:
            return "SET_DUTY_CYCLE";
        default:
            return "NONE";
    }
}

CommandType commandFromText(const String &text)
```

```

{
    if (text == "SET_SLEEP_DURATION")
        return CommandType::SetSleepDuration;
    if (text == "SET_THRESHOLD")
        return CommandType::SetThreshold;
    if (text == "START_OTA")
        return CommandType::StartOta;
    if (text == "SLEEP_NOW")
        return CommandType::SleepNow;
    if (text == "START_PUMP")
        return CommandType::StartPump;
    if (text == "RESEND_CHUNK")
        return CommandType::ResendChunk;
    if (text == "SET_FILTER_MODE")
        return CommandType::SetFilterMode;
    if (text == "SET_PUMP_TIME")
        return CommandType::SetPumpTime;
    if (text == "SET_CONTROL_MODE")
        return CommandType::SetControlMode;
    if (text == "SET_DUTY_CYCLE")
        return CommandType::SetDutyCycle;
    return CommandType::None;
}

String encodeTelemetry(const TelemetryPacket &packet)
{
    String base = "TYPE=DATA";
    base += ",NODE=" + String(packet.nodeId);
    base += ",PID=" + String(packet.packetId);
    base += ",T=" + String(packet.temperatureC, 1);
    base += ",HA=" + String(packet.humidityRh, 1);
    base += ",HS=" + String(packet.soilMoistureVol, 0);
    base += ",VB=" + String(packet.batteryV, 2);
    base += ",ADC=" + String(packet.adcFiltered);
    base += ",SOIL=" + packet.soilStatus;
    base += ",ERR=" + String(packet.errorFlag);
    base += ",CSLEEP=" + String(packet.configSleepMinutes);
    base += ",CTH=" + String(packet.configSoilThresholdVol);
    base += ",CFILTER=" + String(packet.configFilterMode);
    base += ",CPUMP=" + String(packet.configPumpSeconds);
    base += ",CMODE=" + String(packet.configControlMode);
    base += ",CDUTY=" + String(packet.configDutyCycleMode);
    base.toUpperCase();
    base += ",CRC=" + String(crc16Citt(base), HEX);
    base.toUpperCase();
    return base;
}

String encodeAck(const AckPacket &packet)
{
    String base = "TYPE=ACK";
    base += ",NODE=" + String(packet.nodeId);
    base += ",PID=" + String(packet.packetId);
    base += ",OK=" + String(packet.ok ? 1 : 0);
}

```

```
    base += ",CMD=" + commandToText(packet.command);
    base += ",PARAM=" + String(packet.parameter);
    base += ",STATUS=" + packet.status;
    base.toUpperCase();
    base += ",CRC=" + String(crc16Citt(base), HEX);
    base.toUpperCase();
    return base;
}

String encodeOtaChunk(const OtaChunkPacket &packet)
{
    String base = "TYPE=OTA_CHUNK";
    base += ",NODE=" + String(packet.nodeId);
    base += ",OTAID=" + String(packet.otaId);
    base += ",IDX=" + String(packet.chunkIndex);
    base += ",TOTAL=" + String(packet.totalChunks);
    base += ",DATA=" + packet.payloadData;
    base += ",DCRC=" + String(packet.dataCrc, HEX);
    base.toUpperCase();
    base += ",CRC=" + String(crc16Citt(base), HEX);
    base.toUpperCase();
    return base;
}

String encodeOtaStatus(const OtaStatusPacket &packet)
{
    String base = "TYPE=OTA_STATUS";
    base += ",NODE=" + String(packet.nodeId);
    base += ",OTAID=" + String(packet.otaId);
    base += ",IDX=" + String(packet.chunkIndex);
    base += ",OK=" + String(packet.ok ? 1 : 0);
    base += ",STATUS=" + packet.status;
    base.toUpperCase();
    base += ",CRC=" + String(crc16Citt(base), HEX);
    base.toUpperCase();
    return base;
}

bool decodeTelemetry(const String &line, TelemetryPacket &packet)
{
    if (getField(line, "TYPE") != "DATA" || !hasValidCrc(line))
    {
        return false;
    }
    packet.nodeId = uint8_t(getField(line, "NODE").toInt());
    packet.packetId = uint32_t(getField(line, "PID").toInt());
    packet.temperatureC = getField(line, "T").toFloat();
    packet.humidityRh = getField(line, "HA").toFloat();
    packet.soilMoistureVol = getField(line, "HS").toFloat();
    packet.batteryV = getField(line, "VB").toFloat();
    packet.adcFiltered = getField(line, "ADC").toInt();
    packet.soilStatus = getField(line, "SOIL");
    packet.errorFlag = uint8_t(getField(line, "ERR").toInt());
    packet.configSleepMinutes = uint32_t(getField(line, "CSLEEP").toInt());
}
```

```
packet.configSoilThresholdVol = getField(line, "CTH").toInt();
packet.configFilterMode = getField(line, "CFILTER").toInt();
packet.configPumpSeconds = getField(line, "CPUMP").toInt();
packet.configControlMode = getField(line, "CMODE").toInt();
packet.configDutyCycleMode = getField(line, "CDUTY").toInt();
return packet.nodeId > 0 && packet.packetId > 0;
}

bool decodeAck(const String &line, AckPacket &packet)
{
    if (getField(line, "TYPE") != "ACK" || !hasValidCrc(line))
    {
        return false;
    }
    packet.nodeId = uint8_t(getField(line, "NODE").toInt());
    packet.packetId = uint32_t(getField(line, "PID").toInt());
    packet.ok = getField(line, "OK").toInt() == 1;
    packet.command = commandFromText(getField(line, "CMD"));
    packet.parameter = getField(line, "PARAM").toInt();
    packet.status = getField(line, "STATUS");
    return packet.nodeId > 0 && packet.packetId > 0;
}

bool decodeOtaChunk(const String &line, OtaChunkPacket &packet)
{
    if (getField(line, "TYPE") != "OTA_CHUNK" || !hasValidCrc(line))
    {
        return false;
    }
    packet.nodeId = uint8_t(getField(line, "NODE").toInt());
    packet.otaId = uint32_t(getField(line, "OTAID").toInt());
    packet.chunkIndex = uint16_t(getField(line, "IDX").toInt());
    packet.totalChunks = uint16_t(getField(line, "TOTAL").toInt());
    packet.payloadData = getField(line, "DATA");
    packet.dataCrc = uint16_t(strtoul(getField(line, "DCRC").c_str(), nullptr,
16));
    return packet.nodeId > 0 && packet.otaId > 0 && packet.totalChunks > 0;
}

bool decodeOtaStatus(const String &line, OtaStatusPacket &packet)
{
    if (getField(line, "TYPE") != "OTA_STATUS" || !hasValidCrc(line))
    {
        return false;
    }
    packet.nodeId = uint8_t(getField(line, "NODE").toInt());
    packet.otaId = uint32_t(getField(line, "OTAID").toInt());
    packet.chunkIndex = uint16_t(getField(line, "IDX").toInt());
    packet.ok = getField(line, "OK").toInt() == 1;
    packet.status = getField(line, "STATUS");
    return packet.nodeId > 0 && packet.otaId > 0;
}

bool hasValidCrc(const String &line)
```

```

{
    const String expected = getField(line, "CRC");
    if (expected.length() == 0)
    {
        return false;
    }
    const uint16_t actual = crc16Ccitt(withoutCrc(line));
    return uint16_t(strtoul(expected.c_str(), nullptr, 16)) == actual;
}

String getField(const String &line, const String &key)
{
    const String token = key + "=";
    int start = -1;
    if (line.startsWith(token))
    {
        start = 0;
    }
    else
    {
        start = line.indexOf(", " + token);
        if (start >= 0)
        {
            start += 1;
        }
    }

    if (start < 0)
    {
        return "";
    }
    start += token.length();
    int end = line.indexOf(',', start);
    if (end < 0)
    {
        end = line.length();
    }
    return line.substring(start, end);
}

```

src/dht22_sensor.h

```

#pragma once

#include <Arduino.h>

enum class SensorStatus : uint8_t
{
    Ok = 0,
    Error = 1
}

```

```

};

struct DhtReading
{
    float temperatureC = NAN;
    float humidityRh = NAN;
    SensorStatus status = SensorStatus::Error;
    uint8_t errorFlag = 1;
};

class Dht22Sensor
{
public:
    void begin(uint8_t pin, float tempOffsetC, float humidityOffsetRh);
    DhtReading read();

private:
    static constexpr size_t kAverageSize = 5;

    uint8_t _pin = 0;
    float _tempOffsetC = 0.0f;
    float _humidityOffsetRh = 0.0f;
    bool _started = false;
    bool _hasPrevious = false;
    float _previousTemperatureC = 0.0f;
    float _previousHumidityRh = 0.0f;
    float _tempBuffer[kAverageSize] = {};
    float _humidityBuffer[kAverageSize] = {};
    size_t _bufferCount = 0;
    size_t _bufferIndex = 0;

    bool readRaw40Bits(uint8_t bytes[5]);
    bool expectPulse(uint8_t level, uint32_t timeoutUs, uint32_t &durationUs)
const;
    DhtReading fail() const;
    void pushSample(float temperatureC, float humidityRh);
    float average(const float *values) const;
};

```

src/dht22_sensor.cpp

```

#include "dht22_sensor.h"

#include <math.h>

void Dht22Sensor::begin(uint8_t pin, float tempOffsetC, float humidityOffsetRh)
{
    _pin = pin;
    _tempOffsetC = tempOffsetC;
    _humidityOffsetRh = humidityOffsetRh;
}

```

```
    pinMode(_pin, INPUT_PULLUP);
    _started = true;
}

DhtReading Dht22Sensor::read()
{
    if (!_started)
    {
        return fail();
    }

    uint8_t bytes[5] = {};
    if (!readRaw40Bits(bytes))
    {
        return fail();
    }

    const uint8_t checksum = (bytes[0] + bytes[1] + bytes[2] + bytes[3]) & 0xFF;
    if (checksum != bytes[4])
    {
        return fail();
    }

    const uint16_t rawHumidity = (uint16_t(bytes[0]) << 8) | bytes[1];
    uint16_t rawTemperature = (uint16_t(bytes[2]) << 8) | bytes[3];
    const bool negative = (rawTemperature & 0x8000) != 0;
    rawTemperature &= 0x7FFF;

    float humidityRh = rawHumidity / 10.0f + _humidityOffsetRh;
    float temperatureC = rawTemperature / 10.0f + _tempOffsetC;
    if (negative)
    {
        temperatureC = -temperatureC;
    }

    if (temperatureC < -10.0f || temperatureC > 50.0f || humidityRh < 0.0f ||
        humidityRh > 100.0f)
    {
        return fail();
    }

    if (_hasPrevious &&
        (fabsf(temperatureC - _previousTemperatureC) > 2.0f ||
         fabsf(humidityRh - _previousHumidityRh) > 5.0f))
    {
        return fail();
    }

    pushSample(temperatureC, humidityRh);
    _previousTemperatureC = temperatureC;
    _previousHumidityRh = humidityRh;
    _hasPrevious = true;

    DhtReading reading;
```

```
    reading.temperatureC = roundf(average(_tempBuffer) * 10.0f) / 10.0f;
    reading.humidityRh = roundf(average(_humidityBuffer) * 10.0f) / 10.0f;
    reading.status = SensorStatus::Ok;
    reading.errorFlag = 0;
    return reading;
}

bool Dht22Sensor::readRaw40Bits(uint8_t bytes[5])
{
    pinMode(_pin, OUTPUT_OPEN_DRAIN);
    digitalWrite(_pin, LOW);
    delay(20);
    digitalWrite(_pin, HIGH);
    delayMicroseconds(40);
    pinMode(_pin, INPUT_PULLUP);

    uint32_t duration = 0;
    if (!expectPulse(LOW, 100, duration) || !expectPulse(HIGH, 100, duration))
    {
        return false;
    }

    for (uint8_t bitIndex = 0; bitIndex < 40; bitIndex++)
    {
        if (!expectPulse(LOW, 80, duration) || !expectPulse(HIGH, 100, duration))
        {
            return false;
        }

        if (duration > 50)
        {
            bytes[bitIndex / 8] |= (1 << (7 - (bitIndex % 8)));
        }
    }

    return true;
}

bool Dht22Sensor::expectPulse(uint8_t level, uint32_t timeoutUs, uint32_t
&durationUs) const
{
    const uint32_t start = micros();
    while (digitalRead(_pin) != level)
    {
        if (micros() - start > timeoutUs)
        {
            return false;
        }
    }

    const uint32_t pulseStart = micros();
    while (digitalRead(_pin) == level)
    {
        if (micros() - pulseStart > timeoutUs)
```

```

        {
            return false;
        }
    }

    durationUs = micros() - pulseStart;
    return true;
}

DhtReading Dht22Sensor::fail() const
{
    DhtReading reading;
    reading.status = SensorStatus::Error;
    reading.errorFlag = 1;
    return reading;
}

void Dht22Sensor::pushSample(float temperatureC, float humidityRh)
{
    _tempBuffer[_bufferIndex] = temperatureC;
    _humidityBuffer[_bufferIndex] = humidityRh;
    _bufferIndex = (_bufferIndex + 1) % kAverageSize;
    if (_bufferCount < kAverageSize)
    {
        _bufferCount++;
    }
}

float Dht22Sensor::average(const float *values) const
{
    float sum = 0.0f;
    for (size_t i = 0; i < _bufferCount; i++)
    {
        sum += values[i];
    }
    return _bufferCount == 0 ? NAN : sum / _bufferCount;
}

```

src/soil_moisture.h

```

#pragma once

#include <Arduino.h>

#include "dht22_sensor.h"

enum class SoilStatus : uint8_t
{
    UrgentWatering = 0,
    NeedWatering = 1,

```

```

    LightDry = 2,
    Normal = 3,
    OverMoisture = 4,
    Error = 5
};

struct SoilReading
{
    int adcRaw = 0;
    int adcFiltered = 0;
    float moistureVol = NAN;
    SoilStatus soilStatus = SoilStatus::Error;
    SensorStatus sensorStatus = SensorStatus::Error;
    uint8_t errorFlag = 1;
};

class SoilMoistureSensor
{
public:
    void begin(uint8_t adcPin, int adcDry, int adcWet, int adcMin, int adcMax);
    SoilReading read();

    static const char *statusText(SoilStatus status);
    static SoilStatus classify(float moistureVol);

private:
    static constexpr size_t kSamples = 11;
    static constexpr size_t kDiscardFirst = 3;
    static constexpr uint8_t kMaxAdcErrors = 3;

    uint8_t _adcPin = 0;
    int _adcDry = 3400;
    int _adcWet = 1200;
    int _adcMin = 100;
    int _adcMax = 4090;
    uint8_t _adcErrorCount = 0;

    static int median(int *values, size_t length);
    static float clampMoisture(float moistureVol);
};

```

src/soil_moisture.cpp

```

#include "soil_moisture.h"

#include <math.h>

void SoilMoistureSensor::begin(uint8_t adcPin, int adcDry, int adcWet, int adcMin,
int adcMax)
{

```

```

    _adcPin = adcPin;
    _adcDry = adcDry;
    _adcWet = adcWet;
    _adcMin = adcMin;
    _adcMax = adcMax;
    analogReadResolution(12);
    analogSetPinAttenuation(_adcPin, ADC_11db);
}

SoilReading SoilMoistureSensor::read()
{
    SoilReading reading;
    int samples[kSamples - kDiscardFirst] = {};

    for (size_t i = 0; i < kSamples; i++)
    {
        const int adc = analogRead(_adcPin);
        reading.adcRaw = adc;

        if (adc <= _adcMin || adc >= _adcMax)
        {
            _adcErrorCount++;
            if (_adcErrorCount >= kMaxAdcErrors)
            {
                reading.soilStatus = SoilStatus::Error;
                reading.sensorStatus = SensorStatus::Error;
                reading.errorFlag = 1;
                return reading;
            }
            delay(350);
            i--;
            continue;
        }

        if (i >= kDiscardFirst)
        {
            samples[i - kDiscardFirst] = adc;
        }
        delay(20);
    }

    _adcErrorCount = 0;
    reading.adcFiltered = median(samples, kSamples - kDiscardFirst);
    reading.moistureVol = clampMoisture(
        (_adcDry - reading.adcFiltered) * 60.0f / float(_adcDry - _adcWet));
    reading.soilStatus = classify(reading.moistureVol);
    reading.sensorStatus = SensorStatus::Ok;
    reading.errorFlag = 0;
    return reading;
}

const char *SoilMoistureSensor::statusText(SoilStatus status)
{
    switch (status)

```

```
{
    case SoilStatus::UrgentWatering:
        return "URGENT_WATERING";
    case SoilStatus::NeedWatering:
        return "NEED_WATERING";
    case SoilStatus::LightDry:
        return "LIGHT_DRY";
    case SoilStatus::Normal:
        return "NORMAL";
    case SoilStatus::OverMoisture:
        return "OVER_MOISTURE";
    default:
        return "ERROR";
}
}

SoilStatus SoilMoistureSensor::classify(float moistureVol)
{
    if (isnan(moistureVol))
    {
        return SoilStatus::Error;
    }
    if (moistureVol > 42.0f)
    {
        return SoilStatus::OverMoisture;
    }
    if (moistureVol > 33.0f)
    {
        return SoilStatus::Normal;
    }
    if (moistureVol > 24.0f)
    {
        return SoilStatus::LightDry;
    }
    if (moistureVol > 15.0f)
    {
        return SoilStatus::NeedWatering;
    }
    return SoilStatus::UrgentWatering;
}

int SoilMoistureSensor::median(int *values, size_t length)
{
    for (size_t i = 1; i < length; i++)
    {
        const int key = values[i];
        int j = int(i) - 1;
        while (j >= 0 && values[j] > key)
        {
            values[j + 1] = values[j];
            j--;
        }
        values[j + 1] = key;
    }
}
```

```
        return values[length / 2];
    }

    float SoilMoistureSensor::clampMoisture(float moistureVol)
    {
        if (moistureVol < 0.0f)
        {
            return 0.0f;
        }
        if (moistureVol > 60.0f)
        {
            return 60.0f;
        }
        return roundf(moistureVol * 10.0f) / 10.0f;
    }
}
```

src/node_main.cpp

```
#include <Arduino.h>
#include <LoRa.h>
#include <Preferences.h>
#include <SPI.h>

#include "dht22_sensor.h"
#include "packet_protocol.h"
#include "project_config.h"
#include "soil_moisture.h"

RTC_DATA_ATTR uint32_t rtcPacketId = 0;
RTC_DATA_ATTR uint32_t rtcSleepMinutes = Config::DefaultSleepMinutes;

static Dht22Sensor dht22;
static SoilMoistureSensor soilSensor;
static Preferences preferences;

struct NodeRuntimeConfig
{
    int soilThresholdVol = 20;
    uint32_t sleepMinutes = Config::DefaultSleepMinutes;
    int filterMode = 0; // 0=AVERAGE, 1=MEDIAN.
    int pumpSeconds = 5;
    int controlMode = 1; // 0=MANUAL, 1=AUTO.
    int dutyCycleMode = 0; // Fixed 30-minute cycle.
};

static NodeRuntimeConfig runtimeConfig;

static void loadRuntimeConfig()
{
    preferences.begin("node_cfg", true);
```

```
    runtimeConfig.soilThresholdVol = preferences.getInt("soil_th",
runtimeConfig.soilThresholdVol);
    runtimeConfig.filterMode = preferences.getInt("filter",
runtimeConfig.filterMode);
    runtimeConfig.pumpSeconds = preferences.getInt("pump_s",
runtimeConfig.pumpSeconds);
    runtimeConfig.controlMode = preferences.getInt("ctrl",
runtimeConfig.controlMode);
    preferences.end();
    runtimeConfig.sleepMinutes = Config::DefaultSleepMinutes;
    runtimeConfig.dutyCycleMode = 0;
    rtcSleepMinutes = runtimeConfig.sleepMinutes;
}

static void saveRuntimeConfig()
{
    preferences.begin("node_cfg", false);
    preferences.putInt("soil_th", runtimeConfig.soilThresholdVol);
    preferences.putUInt("sleep_min", Config::DefaultSleepMinutes);
    preferences.putInt("filter", runtimeConfig.filterMode);
    preferences.putInt("pump_s", runtimeConfig.pumpSeconds);
    preferences.putInt("ctrl", runtimeConfig.controlMode);
    preferences.putInt("duty", 0);
    preferences.end();
}

static void setPump(bool enabled)
{
    digitalWrite(Config::PumpPin, enabled == Config::PumpActiveHigh ? HIGH : LOW);
}

static bool initLoRa()
{
    LoRa.end();
    SPI.end();
    delay(50);

    pinMode(Config::LoraSs, OUTPUT);
    digitalWrite(Config::LoraSs, HIGH);
    pinMode(Config::LoraRst, OUTPUT);
    digitalWrite(Config::LoraRst, LOW);
    delay(50);
    digitalWrite(Config::LoraRst, HIGH);
    delay(200);

    SPI.begin(Config::LoraSck, Config::LoraMiso, Config::LoraMosi,
Config::LoraSs);
    LoRa.setPins(Config::LoraSs, Config::LoraRst, Config::LoraDio0);
    if (!LoRa.begin(Config::LoraFrequency))
    {
        return false;
    }

    LoRa.setSpreadingFactor(Config::LoraSpreadingFactor);
```

```
    LoRa.setSignalBandwidth(Config::LoraSignalBandwidth);
    LoRa.setCodingRate4(Config::LoraCodingRate);
    LoRa.setTxPower(Config::LoraTxPowerDbm);
    LoRa.enableCrc();
    return true;
}

static float readBatteryVoltage()
{
    analogReadResolution(12);
    analogSetPinAttenuation(Config::BatteryAdcPin, ADC_11db);
    delay(10);

    uint32_t rawSum = 0;
    uint32_t milliVoltSum = 0;
    constexpr uint8_t sampleCount = 16;
    for (uint8_t i = 0; i < sampleCount; i++)
    {
        rawSum += analogRead(Config::BatteryAdcPin);
        milliVoltSum += analogReadMilliVolts(Config::BatteryAdcPin);
        delay(2);
    }

    const float adcVoltage = (milliVoltSum / float(sampleCount)) / 1000.0f;
    return adcVoltage * Config::BatteryDividerRatio;
}

static DhtReading readDht22WithRetry()
{
    constexpr uint8_t maxAttempts = 3;
    for (uint8_t attempt = 1; attempt <= maxAttempts; attempt++)
    {
        const DhtReading reading = dht22.read();
        if (reading.errorFlag == 0)
        {
            return reading;
        }

        Serial.printf("DHT read failed attempt %u/%u\n", attempt, maxAttempts);
        delay(2000);
    }

    return DhtReading{};
}

static void sendOtaStatus(uint32_t otaId, uint16_t chunkIndex, bool ok, const
String &status)
{
    OtaStatusPacket packet;
    packet.nodeId = Config::NodeId;
    packet.otaId = otaId;
    packet.chunkIndex = chunkIndex;
    packet.ok = ok;
    packet.status = status;
}
```

```
    const String payload = encodeOtaStatus(packet);
    LoRa.beginPacket();
    LoRa.print(payload);
    LoRa.endPacket();
    LoRa.receive();
    delay(100);
    Serial.print("OTA STATUS TX: ");
    Serial.println(payload);
}

static bool waitForOtaChunk(uint32_t otaId, uint16_t expectedIndex, OtaChunkPacket
&chunk,
                            bool advertiseReady)
{
    const uint32_t start = millis();
    uint32_t lastReadyMs = 0;
    while (millis() - start < 12000)
    {
        if (advertiseReady && millis() - lastReadyMs >= 1200)
        {
            lastReadyMs = millis();
            sendOtaStatus(otaId, 0, true, "OTA_READY");
        }

        const int packetSize = LoRa.parsePacket();
        if (packetSize <= 0)
        {
            delay(10);
            continue;
        }

        String line;
        while (LoRa.available())
        {
            line += char(LoRa.read());
        }

        if (!decodeOtaChunk(line, chunk))
        {
            Serial.print("OTA chunk invalid: ");
            Serial.println(line);
            sendOtaStatus(otaId, expectedIndex, false, "NACK_BAD_PACKET");
            continue;
        }
        if (chunk.nodeId != Config::NodeId || chunk.otaId != otaId ||
chunk.chunkIndex != expectedIndex)
        {
            Serial.println("OTA chunk ignored: node/ota/index mismatch");
            continue;
        }
        return true;
    }
    return false;
}
```

```

}

static void runSimulatedLoraOta(uint32_t otaId)
{
    if (otaId == 0)
    {
        otaId = rtcPacketId;
    }

    Serial.printf("START_OTA session ota_id=%lu\n", otaId);
    Serial.println("OTA simulation only; firmware flash is not modified");

    uint16_t expectedTotal = 0;
    uint16_t receivedChunks = 0;
    uint16_t firmwareCrc = 0xFFFF;

    while (true)
    {
        OtaChunkPacket chunk;
        const bool advertiseReady = receivedChunks == 0;
        if (!waitForOtaChunk(otaId, receivedChunks + 1, chunk, advertiseReady))
        {
            sendOtaStatus(otaId, receivedChunks + 1, false, "OTA_FAILED_TIMEOUT");
            return;
        }

        const uint16_t actualDataCrc = crc16Ccitt(chunk.payloadData);
        if (actualDataCrc != chunk.dataCrc)
        {
            Serial.printf("OTA chunk CRC fail idx=%u expected=%04X actual=%04X\n",
                           chunk.chunkIndex, chunk.dataCrc, actualDataCrc);
            sendOtaStatus(otaId, chunk.chunkIndex, false, "NACK_CRC");
            continue;
        }

        if (expectedTotal == 0)
        {
            expectedTotal = chunk.totalChunks;
        }
        firmwareCrc = crc16Ccitt(reinterpret_cast<const uint8_t*>
                                   (chunk.payloadData.c_str()),
                                   chunk.payloadData.length()) ^ firmwareCrc;
        receivedChunks++;
        sendOtaStatus(otaId, chunk.chunkIndex, true, "ACK");

        if (receivedChunks >= expectedTotal)
        {
            Serial.printf("OTA simulated firmware CRC=%04X chunks=%u\n",
                           firmwareCrc, receivedChunks);
            delay(300);
            for (uint8_t i = 0; i < 3; i++)
            {
                sendOtaStatus(otaId, receivedChunks, true, "OTA_SUCCESS");
                delay(300);
            }
        }
    }
}

```

```

    }
    return;
}
}
}

static void executeCommand(const AckPacket &ack)
{
    if (!ack.ok)
    {
        return;
    }

    switch (ack.command)
    {
    case CommandType::SetSleepDuration:
        rtcSleepMinutes = Config::DefaultSleepMinutes;
        runtimeConfig.sleepMinutes = Config::DefaultSleepMinutes;
        saveRuntimeConfig();
        Serial.printf("Command SET_SLEEP_DURATION ignored: fixed %lu min\n",
rtcSleepMinutes);
        break;
    case CommandType::SetThreshold:
        runtimeConfig.soilThresholdVol = constrain(ack.parameter, 0, 100);
        saveRuntimeConfig();
        Serial.printf("Command SET_THRESHOLD: soil_min=%d %%Vol\n",
runtimeConfig.soilThresholdVol);
        break;
    case CommandType::SetFilterMode:
        runtimeConfig.filterMode = constrain(ack.parameter, 0, 1);
        saveRuntimeConfig();
        Serial.printf("Command SET_FILTER_MODE: %s\n", runtimeConfig.filterMode ==
1 ? "MEDIAN" : "AVERAGE");
        break;
    case CommandType::SetPumpTime:
        runtimeConfig.pumpSeconds = constrain(ack.parameter, 0,
int(Config::MaxPumpSeconds));
        saveRuntimeConfig();
        Serial.printf("Command SET_PUMP_TIME: %d s\n", runtimeConfig.pumpSeconds);
        break;
    case CommandType::SetControlMode:
        runtimeConfig.controlMode = constrain(ack.parameter, 0, 1);
        saveRuntimeConfig();
        Serial.printf("Command SET_CONTROL_MODE: %s\n", runtimeConfig.controlMode
== 1 ? "AUTO" : "MANUAL");
        break;
    case CommandType::SetDutyCycle:
        runtimeConfig.dutyCycleMode = 0;
        saveRuntimeConfig();
        Serial.println("Command SET_DUTY_CYCLE ignored: FIXED");
        break;
    case CommandType::SleepNow:
        Serial.println("Command SLEEP_NOW");
        break;
    }
}

```

```
    case CommandType::StartPump:
    {
        const int pumpSeconds = constrain(ack.parameter, 0,
int(Config::MaxPumpSeconds));
        Serial.printf("Command START_PUMP: %d s\n", pumpSeconds);
        if (pumpSeconds > 0)
        {
            setPump(true);
            delay(uint32_t(pumpSeconds) * 1000UL);
            setPump(false);
            Serial.println("Pump OFF");
        }
        break;
    }
    case CommandType::StartOta:
        runSimulatedLoraOta(uint32_t(ack.parameter));
        break;
    default:
        break;
}

static bool waitForAck(uint32_t packetId)
{
    const uint32_t start = millis();
    while (millis() - start < Config::AckTimeoutMs)
    {
        const int packetSize = LoRa.parsePacket();
        if (packetSize <= 0)
        {
            delay(10);
            continue;
        }

        String line;
        while (LoRa.available())
        {
            line += char(LoRa.read());
        }

        AckPacket ack;
        if (!decodeAck(line, ack))
        {
            Serial.println("ACK invalid CRC/header");
            Serial.print("ACK raw: ");
            Serial.println(line);
            continue;
        }
        if (ack.nodeId != Config::NodeId || ack.packetId != packetId)
        {
            Serial.println("ACK ignored: Node_ID/Package_ID mismatch");
            continue;
        }
    }
}
```

```
        Serial.print("ACK received: ");
        Serial.println(line);
        executeCommand(ack);
        return ack.ok;
    }
    return false;
}

static bool sendTelemetry(const TelemetryPacket &telemetry)
{
    const String payload = encodeTelemetry(telemetry);
    Serial.print("LoRa TX: ");
    Serial.println(payload);

    LoRa.beginPacket();
    LoRa.print(payload);
    LoRa.endPacket(true);
    delay(250);
    LoRa.receive();

    return waitForAck(telemetry.packetId);
}

static void enterSleepSeconds(uint32_t sleepSeconds)
{
    sleepSeconds = constrain(sleepSeconds, 5UL, 90UL * 60UL);
    rtcSleepMinutes = max(1UL, sleepSeconds / 60UL);
    LoRa.sleep();
    LoRa.end();
    SPI.end();
    digitalWrite(Config::SensorPowerPin, LOW);

    if (sleepSeconds < 60)
    {
        Serial.printf("Sleep duration: %lu sec\n", sleepSeconds);
    }
    else
    {
        Serial.printf("Sleep duration: %lu min\n", sleepSeconds / 60UL);
    }
    Serial.flush();

    if (Config::EnableDeepSleep)
    {
        esp_sleep_enable_timer_wakeup(uint64_t(sleepSeconds) * 1000000ULL);
        esp_deep_sleep_start();
    }
}

static void enterSleep(uint32_t sleepMinutes)
{
    enterSleepSeconds(sleepMinutes * 60UL);
}
```

```
void setup()
{
    Serial.begin(Config::SerialBaud);
    delay(1000);

    pinMode(Config::SensorPowerPin, OUTPUT);
    digitalWrite(Config::SensorPowerPin, HIGH);
    pinMode(Config::PumpPin, OUTPUT);
    setPump(false);
    delay(2000);

    Serial.println("==== EE4552 NODE FIRMWARE =====");
    Serial.printf("Wake cause: %d\n", int(esp_sleep_get_wakeup_cause()));
    loadRuntimeConfig();
    Serial.printf("Config: soil_threshold=%d sleep=%lu filter=%d pump=%d
control=%d duty=%d\n",
        runtimeConfig.soilThresholdVol, runtimeConfig.sleepMinutes,
        runtimeConfig.filterMode, runtimeConfig.pumpSeconds,
        runtimeConfig.controlMode, runtimeConfig.dutyCycleMode);

    dht22.begin(Config::DhtPin, Config::TempOffsetC, Config::HumidityOffsetRh);
    soilSensor.begin(Config::SoilAdcPin, Config::SoilAdcDry, Config::SoilAdcWet,
        Config::SoilAdcMin, Config::SoilAdcMax);

    if (!initLoRa())
    {
        const uint32_t failureSleepMinutes = Config::DefaultSleepMinutes;
        Serial.printf("LoRa init failed; sleeping %lu min\n",
failureSleepMinutes);
        enterSleep(failureSleepMinutes);
        return;
    }
    Serial.println("LoRa init OK");

    const DhtReading dht = readDht22WithRetry();
    const SoilReading soil = soilSensor.read();
    const float batteryV = readBatteryVoltage();

    TelemetryPacket telemetry;
    telemetry.nodeId = Config::NodeId;
    telemetry.packetId = ++rtcPacketId;
    telemetry.temperatureC = dht.temperatureC;
    telemetry.humidityRh = dht.humidityRh;
    telemetry.soilMoistureVol = soil.moistureVol;
    telemetry.batteryV = batteryV;
    telemetry.adcFiltered = soil.adcFiltered;
    telemetry.errorFlag = dht.errorFlag || soil.errorFlag;
    telemetry.soilStatus = SoilMoistureSensor::statusText(soil.soilStatus);
    telemetry.configSleepMinutes = runtimeConfig.sleepMinutes;
    telemetry.configSoilThresholdVol = runtimeConfig.soilThresholdVol;
    telemetry.configFilterMode = runtimeConfig.filterMode;
    telemetry.configPumpSeconds = runtimeConfig.pumpSeconds;
    telemetry.configControlMode = runtimeConfig.controlMode;
    telemetry.configDutyCycleMode = runtimeConfig.dutyCycleMode;
```

```

    Serial.printf("DHT: T=%.1f C H=%.1f %%RH ERR=%u\n", telemetry.temperatureC,
                  telemetry.humidityRh, dht.errorFlag);
    Serial.printf("SOIL: ADC=%d H=%.1f %%Vol status=%s ERR=%u\n",
telemetry.adcFiltered,
                  telemetry.soilMoistureVol, telemetry.soilStatus.c_str(),
soil.errorFlag);
    Serial.printf("Battery: %.2f V\n", telemetry.batteryV);

    const bool delivered = sendTelemetry(telemetry);
    if (!delivered)
    {
        Serial.println("No ACK; TX failed");
    }

    enterSleep(Config::DefaultSleepMinutes);
}

void loop()
{
    if (!Config::EnableDeepSleep)
    {
        delay(uint32_t(rtcSleepMinutes) * 60UL * 1000UL);
        ESP.restart();
    }
}

```

src/gateway_state.h

```

#pragma once

#include <Arduino.h>

#include "packet_protocol.h"

struct StoredTelemetry
{
    TelemetryPacket packet;
    int rssi = 0;
    float snr = 0.0f;
    uint32_t receivedMs = 0;
};

constexpr size_t kGatewayHistorySize = 64;

extern StoredTelemetry history[kGatewayHistorySize];
extern size_t historyCount;
extern size_t historyWriteIndex;
extern uint32_t lastPacketIdByNode[256];
extern AckPacket pendingCommands[256];

```

```
extern uint8_t pendingCommandRepeats[256];
extern portMUX_TYPE pendingCommandMux;
extern uint32_t lastHeartbeatMs;
extern uint32_t lastLoraRetryMs;
extern bool loraReady;
extern bool otaSessionActive;
extern bool otaReadyWaitActive;
extern uint8_t otaReadyWaitNodeId;
extern uint32_t otaReadyWaitId;
extern uint32_t otaReadyWaitDeadlineMs;
```

src/gateway_state.cpp

```
#include "gateway_state.h"

StoredTelemetry history[kGatewayHistorySize];
size_t historyCount = 0;
size_t historyWriteIndex = 0;
uint32_t lastPacketIdByNode[256] = {};
AckPacket pendingCommands[256];
uint8_t pendingCommandRepeats[256] = {};
portMUX_TYPE pendingCommandMux = portMUX_INITIALIZER_UNLOCKED;
uint32_t lastHeartbeatMs = 0;
uint32_t lastLoraRetryMs = 0;
bool loraReady = false;
bool otaSessionActive = false;
bool otaReadyWaitActive = false;
uint8_t otaReadyWaitNodeId = 0;
uint32_t otaReadyWaitId = 0;
uint32_t otaReadyWaitDeadlineMs = 0;
```

src/gateway_logic.h

```
#pragma once

#include <Arduino.h>

#include "packet_protocol.h"

int parseCommandParameter(CommandType command, String value);
int computePumpTime(const TelemetryPacket &packet);
String alertLevel(const TelemetryPacket &packet);
void storeTelemetry(const TelemetryPacket &packet, int rssi, float snr);
void queueGatewayCommand(uint8_t nodeId, CommandType command, int parameter);
```

src/gateway_logic.cpp

```
#include "gateway_logic.h"

#include "gateway_state.h"

int parseCommandParameter(CommandType command, String value)
{
    value.trim();
    value.toUpperCase();
    if (command == CommandType::SetFilterMode)
    {
        return value == "MEDIAN" ? 1 : 0;
    }
    if (command == CommandType::SetControlMode)
    {
        return value == "AUTO" ? 1 : 0;
    }
    if (command == CommandType::SetDutyCycle)
    {
        return value == "ADAPTIVE" ? 1 : 0;
    }
    return value.toInt();
}

int computePumpTime(const TelemetryPacket &packet)
{
    if (packet.errorFlag != 0)
    {
        return 0;
    }

    int pumpTime = 0;
    if (packet.soilMoistureVol > 42.0f)
    {
        pumpTime = 0;
    }
    else if (packet.soilMoistureVol > 33.0f)
    {
        pumpTime = 0;
    }
    else if (packet.soilMoistureVol > 24.0f)
    {
        pumpTime = 3;
    }
    else if (packet.soilMoistureVol > 15.0f)
    {
        pumpTime = 6;
    }
    else
    {
        pumpTime = 10;
    }
}
```

```
    }

    if (packet.temperatureC > 32.0f)
    {
        pumpTime += 2;
    }
    if (packet.humidityRh < 50.0f)
    {
        pumpTime += 3;
    }
    if (packet.humidityRh > 80.0f)
    {
        pumpTime -= 2;
    }
    return constrain(pumpTime, 0, 15);
}

String alertLevel(const TelemetryPacket &packet)
{
    if (packet.errorFlag != 0)
    {
        return "SENSOR_ERROR";
    }
    if (packet.soilMoistureVol > 42.0f)
    {
        return "OVER_MOISTURE";
    }
    if (packet.soilMoistureVol > 33.0f)
    {
        return "NORMAL";
    }
    if (packet.soilMoistureVol > 24.0f)
    {
        return "LIGHT_DRY";
    }
    if (packet.soilMoistureVol > 15.0f)
    {
        return "NEED_WATERING";
    }
    return "URGENT_WATERING";
}

void storeTelemetry(const TelemetryPacket &packet, int rssi, float snr)
{
    history[historyWriteIndex].packet = packet;
    history[historyWriteIndex].rssi = rssi;
    history[historyWriteIndex].snr = snr;
    history[historyWriteIndex].receivedMs = millis();
    historyWriteIndex = (historyWriteIndex + 1) % kGatewayHistorySize;
    if (historyCount < kGatewayHistorySize)
    {
        historyCount++;
    }
}
```

```
void queueGatewayCommand(uint8_t nodeId, CommandType command, int parameter)
{
    portENTER_CRITICAL(&pendingCommandMux);
    pendingCommands[nodeId].nodeId = nodeId;
    pendingCommands[nodeId].command = command;
    pendingCommands[nodeId].parameter = parameter;
    pendingCommands[nodeId].ok = true;
    pendingCommands[nodeId].status = "QUEUED";
    pendingCommandRepeats[nodeId] = command == CommandType::StartOta ? 10 : 3;
    portEXIT_CRITICAL(&pendingCommandMux);
}
```

src/gateway_lora.h

```
#pragma once

bool initLoRa();
void processLoRa();
void processSerialCommand();
```

src/gateway_lora.cpp

```
#include "gateway_lora.h"

#include <Arduino.h>
#include <LoRa.h>
#include <SPI.h>

#include "gateway_cloud.h"
#include "gateway_logic.h"
#include "gateway_state.h"
#include "packet_protocol.h"
#include "project_config.h"

static void sendAckFor(const TelemetryPacket &packet);

bool initLoRa()
{
    LoRa.end();
    SPI.end();
    delay(50);

    pinMode(Config::LoraSs, OUTPUT);
    digitalWrite(Config::LoraSs, HIGH);
    pinMode(Config::LoraRst, OUTPUT);
    digitalWrite(Config::LoraRst, LOW);
}
```

```
    delay(50);
    digitalWrite(Config::LoraRst, HIGH);
    delay(200);

    SPI.begin(Config::LoraSck, Config::LoraMiso, Config::LoraMosi,
Config::LoraSs);
    LoRa.setPins(Config::LoraSs, Config::LoraRst, Config::LoraDio0);
    if (!LoRa.begin(Config::LoraFrequency))
    {
        LoRa.end();
        SPI.end();
        return false;
    }

    LoRa.setSpreadingFactor(Config::LoraSpreadingFactor);
    LoRa.setSignalBandwidth(Config::LoraSignalBandwidth);
    LoRa.setCodingRate4(Config::LoraCodingRate);
    LoRa.setTxPower(Config::LoraTxPowerDbm);
    LoRa.enableCrc();
    LoRa.receive();
    return true;
}

static bool waitForOtaStatus(uint8_t nodeId, uint32_t otaId, uint16_t chunkIndex,
                             const String &expectedStatus, OtaStatusPacket
&status)
{
    const uint32_t start = millis();
    while (millis() - start < 7000)
    {
        const int packetSize = LoRa.parsePacket();
        if (packetSize <= 0)
        {
            delay(10);
            continue;
        }

        String line;
        while (LoRa.available())
        {
            line += char(LoRa.read());
        }

        if (!decodeOtaStatus(line, status))
        {
            TelemetryPacket telemetry;
            if (chunkIndex == 0 && decodeTelemetry(line, telemetry) &&
telemetry.nodeId == nodeId)
            {
                Serial.println("OTA wait received DATA; resending START_OTA ACK");
                sendAckFor(telemetry);
                continue;
            }
            Serial.print("OTA status invalid: ");

```

```

        Serial.println(line);
        continue;
    }
    if (status.nodeId != nodeId || status.otaId != otaId)
    {
        Serial.println("OTA status ignored: node/ota mismatch");
        continue;
    }
    if (status.chunkIndex != chunkIndex && expectedStatus != "OTA_SUCCESS")
    {
        Serial.printf("OTA status ignored: chunk mismatch idx=%u expected=%u
status=%s\n",
                        status.chunkIndex, chunkIndex, status.status.c_str());
        continue;
    }
    if (expectedStatus.length() > 0 && status.status != expectedStatus)
    {
        Serial.printf("OTA status unexpected: %s\n", status.status.c_str());
        return false;
    }
    return status.ok;
}
return false;
}

static bool sendOtaChunk(uint8_t nodeId, uint32_t otaId, uint16_t index,
                        uint16_t total, const String &data)
{
    constexpr uint8_t maxAttempts = 3;
    for (uint8_t attempt = 1; attempt <= maxAttempts; attempt++)
    {
        OtaChunkPacket chunk;
        chunk.nodeId = nodeId;
        chunk.otaId = otaId;
        chunk.chunkIndex = index;
        chunk.totalChunks = total;
        chunk.payloadData = data;
        chunk.dataCrc = crc16Ccitt(data);

        const String payload = encodeOtaChunk(chunk);
        Serial.printf("OTA CHUNK TX idx=%u try=%u: %s\n", index, attempt,
payload.c_str());
        LoRa.beginPacket();
        LoRa.print(payload);
        LoRa.endPacket();
        LoRa.receive();
        delay(150);

        OtaStatusPacket status;
        if (waitForOtaStatus(nodeId, otaId, index, "ACK", status))
        {
            return true;
        }
        Serial.printf("OTA chunk retry idx=%u next_try=%u\n", index, attempt + 1);
    }
}

```

```
    }
    return false;
}

static void runSimulatedLoraOtaTransfer(uint8_t nodeId, uint32_t otaId)
{
    if (otaSessionActive)
    {
        return;
    }
    otaSessionActive = true;

    static const char *chunks[] = {
        "FW_SIM_001_BOOT_HDR",
        "FW_SIM_002_PARTITION",
        "FW_SIM_003_APP_TEXT",
        "FW_SIM_004_APP_DATA",
        "FW_SIM_005_DRIVER",
        "FW_SIM_006_SENSOR",
        "FW_SIM_007_LORA",
        "FW_SIM_008_CONFIG",
        "FW_SIM_009_CHECKSUM",
        "FW_SIM_010_END"};

    Serial.printf("OTA session start node=%u ota_id=%lu\n", nodeId, otaId);
    LoRa.receive();
    delay(500);

    constexpr uint16_t totalChunks = sizeof(chunks) / sizeof(chunks[0]);
    for (uint16_t i = 0; i < totalChunks; i++)
    {
        if (!sendOtaChunk(nodeId, otaId, i + 1, totalChunks, chunks[i]))
        {
            Serial.printf("OTA session failed at chunk %u\n", i + 1);
            otaSessionActive = false;
            return;
        }
    }

    OtaStatusPacket done;
    if (waitForOtaStatus(nodeId, otaId, totalChunks, "OTA_SUCCESS", done))
    {
        Serial.printf("OTA session success node=%u ota_id=%lu chunks=%u\n",
nodeId, otaId, totalChunks);
    }
    else
    {
        Serial.println("OTA session finished chunks but final OTA_SUCCESS not
received");
    }
    otaSessionActive = false;
}

static void sendAckFor(const TelemetryPacket &packet)
```

```
{
    if (!loraReady)
    {
        return;
    }

    AckPacket ack;
    ack.nodeId = packet.nodeId;
    ack.packetId = packet.packetId;
    ack.ok = packet.errorFlag == 0;
    ack.status = ack.ok ? "OK" : "SENSOR_ERROR";

    CommandType queuedCommand = CommandType::None;
    int queuedParameter = 0;
    portENTER_CRITICAL(&pendingCommandMux);
    queuedCommand = pendingCommands[packet.nodeId].command;
    queuedParameter = pendingCommands[packet.nodeId].parameter;
    if (queuedCommand != CommandType::None)
    {
        if (pendingCommandRepeats[packet.nodeId] > 0)
        {
            pendingCommandRepeats[packet.nodeId]--;
        }
        if (pendingCommandRepeats[packet.nodeId] == 0)
        {
            pendingCommands[packet.nodeId].command = CommandType::None;
            pendingCommands[packet.nodeId].parameter = 0;
            pendingCommands[packet.nodeId].status = "SENT";
        }
    }
    portEXIT_CRITICAL(&pendingCommandMux);

    if (queuedCommand != CommandType::None)
    {
        ack.command = queuedCommand;
        ack.parameter = queuedParameter;
        ack.status = "COMMAND";
    }
    else if (Config::EnablePumpHardware)
    {
        const int pumpTime = computePumpTime(packet);
        if (pumpTime > 0)
        {
            ack.command = CommandType::StartPump;
            ack.parameter = pumpTime;
        }
    }

    const String payload = encodeAck(ack);
    LoRa.beginPacket();
    LoRa.print(payload);
    LoRa.endPacket();
    LoRa.receive();
    delay(100);
}
```

```
Serial.print("ACK TX: ");
Serial.println(payload);

if (queuedCommand == CommandType::StartOta)
{
    otaReadyWaitActive = true;
    otaReadyWaitNodeId = packet.nodeId;
    otaReadyWaitId = uint32_t(ack.parameter);
    otaReadyWaitDeadlineMs = millis() + 30000UL;
    Serial.printf("OTA waiting for OTA_READY node=%u ota_id=%lu\n",
otaReadyWaitNodeId, otaReadyWaitId);
}
}

void processLoRa()
{
    if (!loRaReady)
    {
        return;
    }

    const int packetSize = LoRa.parsePacket();
    if (packetSize <= 0)
    {
        return;
    }

    String line;
    while (LoRa.available())
    {
        line += char(LoRa.read());
    }

    OtaStatusPacket otaStatus;
    if (decodeOtaStatus(line, otaStatus))
    {
        Serial.printf("OTA STATUS RX node=%u ota_id=%lu idx=%u ok=%u status=%s\n",
            otaStatus.nodeId, otaStatus.otaId, otaStatus.chunkIndex,
            otaStatus.ok ? 1 : 0, otaStatus.status.c_str());
        if (otaReadyWaitActive && otaStatus.nodeId == otaReadyWaitNodeId &&
            otaStatus.otaId == otaReadyWaitId && otaStatus.status == "OTA_READY"
&& otaStatus.ok)
        {
            otaReadyWaitActive = false;
            runSimulatedLoraOtaTransfer(otaStatus.nodeId, otaStatus.otaId);
        }
        return;
    }

    TelemetryPacket packet;
    if (!decodeTelemetry(line, packet))
    {
        Serial.print("Invalid RF packet: ");
        Serial.println(line);
    }
}
```

```

        return;
    }

    if (lastPacketIdByNode[packet.nodeId] == packet.packetId)
    {
        Serial.printf("Duplicate packet ignored: node=%u pid=%lu\n",
packet.nodeId, packet.packetId);
        sendAckFor(packet);
        return;
    }
    lastPacketIdByNode[packet.nodeId] = packet.packetId;

    const int rssi = LoRa.packetRssi();
    const float snr = LoRa.packetSnr();
    storeTelemetry(packet, rssi, snr);

    Serial.printf("DATA RX node=%u pid=%lu T=%.1f H=%.1f Soil=%.0f ADC=%d V=%.2f
RSSI=%d SNR=%.1f Alert=%s PumpTime=%d\n",
        packet.nodeId, packet.packetId, packet.temperatureC,
packet.humidityRh,
        packet.soilMoistureVol, packet.adcFiltered, packet.batteryV,
rssi, snr,
        alertLevel(packet).c_str(), computePumpTime(packet));

    uploadThingsBoard(packet, rssi);
    sendAckFor(packet);
}

void processSerialCommand()
{
    if (!Serial.available())
    {
        return;
    }

    String line = Serial.readStringUntil('\n');
    line.trim();
    line.toUpperCase();

    const int firstSpace = line.indexOf(' ');
    const int secondSpace = line.indexOf(' ', firstSpace + 1);
    if (firstSpace < 0)
    {
        Serial.println("Serial command format: NODE_ID CMD PARAM");
        return;
    }

    const uint8_t nodeId = uint8_t(line.substring(0, firstSpace).toInt());
    const String cmdText = secondSpace < 0 ? line.substring(firstSpace + 1)
        : line.substring(firstSpace + 1,
secondSpace);
    const int parameter = secondSpace < 0 ? 0 : line.substring(secondSpace +
1).toInt();
    const CommandType command = commandFromText(cmdText);

```

```
if (nodeId == 0 || command == CommandType::None)
{
    Serial.println("Invalid serial command. Example: 1 START_PUMP 5");
    return;
}

queueGatewayCommand(nodeId, command, parameter);
Serial.printf("Queued command for node %u: %s %d\n", nodeId, cmdText.c_str(),
parameter);
}
```

src/gateway_cloud.h

```
#pragma once

#include "packet_protocol.h"

void uploadThingsBoard(const TelemetryPacket &packet, int rssi);
```

src/gateway_cloud.cpp

```
#include "gateway_cloud.h"

#include <Arduino.h>
#include <HTTPClient.h>
#include <WiFi.h>

#include "gateway_logic.h"
#include "project_config.h"

void uploadThingsBoard(const TelemetryPacket &packet, int rssi)
{
    if (!Config::EnableCloudUpload || WiFi.status() != WL_CONNECTED ||
        String(Config::ThingsBoardToken).length() == 0)
    {
        return;
    }

    HTTPClient http;
    http.setTimeout(1500);
    const String url = String("http://") + Config::ThingsBoardHost + ":" +
        String(Config::ThingsBoardHttpPort) + "/api/v1/" +
        Config::ThingsBoardToken + "/telemetry";

    http.begin(url);
    http.addHeader("Content-Type", "application/json");
    const String body = String("{\"Node_ID\":") + packet.nodeId +
        "\",\"T_air\":") + String(packet.temperatureC, 1) +
```

```

        ", \"H_air\": \"\" + String(packet.humidityRh, 1) +
        ", \"H_soil\": \"\" + String(packet.soilMoistureVol, 0) +
        ", \"V_bat\": \"\" + String(packet.batteryV, 2) +
        ", \"RSSI\": \"\" + String(rssi) +
        ", \"Error_Flag\": \"\" + String(packet.errorFlag) +
        ", \"Alert\": \"\" + alertLevel(packet) + \"\" +
        ", \"PumpTime\": \"\" + String(computePumpTime(packet)) +
        ", \"Sleep_Min\": \"\" + String(packet.configSleepMinutes) +
        ", \"Soil_Threshold\": \"\" +
String(packet.configSoilThresholdVol) +
        ", \"Filter_Mode\": \"\" + String(packet.configFilterMode) +
        ", \"Config_Pump_Time\": \"\" +
String(packet.configPumpSeconds) +
        ", \"Control_Mode\": \"\" + String(packet.configControlMode) +
        ", \"Duty_Cycle\": \"\" + String(packet.configDutyCycleMode) +
    \"}\";
    const int status = http.POST(body);
    Serial.printf("ThingsBoard HTTP status: %d\\n", status);
    http.end();
}

```

src/gateway_web.h

```

#pragma once

void connectWiFi();
void setupWebServer();
void startWebServerTask();

```

src/gateway_web.cpp

```

#include "gateway_web.h"

#include <Arduino.h>
#include <HTTPClient.h>
#include <Update.h>
#include <WebServer.h>
#include <WiFi.h>

#include "gateway_logic.h"
#include "gateway_state.h"
#include "packet_protocol.h"
#include "project_config.h"

static WebServer server(80);
static WiFiServer debugServer(8080);
static TaskHandle_t webTaskHandle = nullptr;

```

```

static void handleDebugServer();

void connectWiFi()
{
    if (String(Config::WifiSsid) == "TEN_WIFI_CUA_BAN")
    {
        Serial.println("WiFi disabled: set WifiSsid/WifiPassword in
project_config.h");
        return;
    }

    WiFi.mode(WIFI_STA);
    WiFi.setSleep(false);
    WiFi.begin(Config::WifiSsid, Config::WifiPassword);
    Serial.print("Connecting WiFi");
    const uint32_t start = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - start < 15000)
    {
        Serial.print(".");
        delay(500);
    }
    Serial.println();
    if (WiFi.status() == WL_CONNECTED)
    {
        Serial.print("Gateway IP: ");
        Serial.println(WiFi.localIP());
    }
}

static String csvHistory()
{
    String csv =
"time_ms,node_id,packet_id,t_air,h_air,h_soil,v_bat,adc,soil_status,error_flag,rss
i,snr,sleep_min,soil_threshold,filter_mode,pump_time,control_mode,duty_cycle\n";
    for (size_t i = 0; i < historyCount; i++)
    {
        const size_t idx = (historyWriteIndex + kGatewayHistorySize - historyCount
+ i) % kGatewayHistorySize;
        const StoredTelemetry &item = history[idx];
        csv += String(item.receivedMs) + "," + item.packet.nodeId + "," +
item.packet.packetId + ",";
        csv += String(item.packet.temperatureC, 1) + "," +
String(item.packet.humidityRh, 1) + ",";
        csv += String(item.packet.soilMoistureVol, 1) + "," +
String(item.packet.batteryV, 2) + ",";
        csv += String(item.packet.adcFiltered) + "," + item.packet.soilStatus +
",";
        csv += String(item.packet.errorFlag) + "," + String(item.rssi) + "," +
String(item.snr, 1) + ",";
        csv += String(item.packet.configSleepMinutes) + "," +
String(item.packet.configSoilThresholdVol) + ",";
        csv += String(item.packet.configFilterMode) + "," +
String(item.packet.configPumpSeconds) + ",";
    }
}

```

```

        csv += String(item.packet.configControlMode) + "," +
String(item.packet.configDutyCycleMode) + "\n";
    }
    return csv;
}

static void handleRoot()
{
    if (Config::EnableGatewayHttpDebug)
    {
        Serial.println("HTTP GET /");
    }
    String html = "<!doctype html><html><head><meta charset='utf-8'><meta
name='viewport' content='width=device-width,initial-scale=1'>";
    html += "<title>EE4552 Gateway</title><style>body{font-
family:Arial;margin:24px}table{border-
collapse:collapse;width:100%}td,th{border:1px solid #ccc;padding:6px;text-
align:left}</style></head><body>";
    html += "<h2>EE4552 Gateway Dashboard</h2><p><a href='/export.csv'>Export
CSV</a></p>";
    html += "<p>Firmware: " + String(Config::GatewayFirmwareVersion) + "</p>";
    html += "<p>Queue command: /command?node=1&cmd=SET_THRESHOLD&param=20
or SET_FILTER_MODE, SET_PUMP_TIME, SET_CONTROL_MODE, START_OTA</p>";
    html += "<p>Gateway OTA: /self_ota?
url=http://server/firmware.bin&md5=optional_md5</p>";
    html += "<table><tr><th>ms</th><th>Node</th><th>PID</th><th>T</th>
<th>H_air</th><th>H_soil</th><th>Vbat</th><th>Status</th><th>RSSI</th>
<th>Sleep</th><th>Threshold</th><th>Pump</th><th>Control</th><th>Duty</th></tr>";
    for (size_t i = 0; i < historyCount; i++)
    {
        const size_t idx = (historyWriteIndex + kGatewayHistorySize - historyCount
+ i) % kGatewayHistorySize;
        const StoredTelemetry &item = history[idx];
        html += "<tr><td>" + String(item.receivedMs) + "</td><td>" +
String(item.packet.nodeId) + "</td><td>" + String(item.packet.packetId) + "</td>";
        html += "<td>" + String(item.packet.temperatureC, 1) + "</td><td>" +
String(item.packet.humidityRh, 1) + "</td><td>" +
String(item.packet.soilMoistureVol, 1) + "</td>";
        html += "<td>" + String(item.packet.batteryV, 2) + "</td><td>" +
alertLevel(item.packet) + "</td><td>" + String(item.rssi) + "</td>";
        html += "<td>" + String(item.packet.configSleepMinutes) + "</td><td>" +
String(item.packet.configSoilThresholdVol) + "</td>";
        html += "<td>" + String(item.packet.configPumpSeconds) + "</td><td>" +
String(item.packet.configControlMode) + "</td>";
        html += "<td>" + String(item.packet.configDutyCycleMode) + "</td></tr>";
    }
    html += "</table></body></html>";
    server.send(200, "text/html", html);
}

static void handleExport()
{
    if (Config::EnableGatewayHttpDebug)
    {

```

```
        Serial.println("HTTP GET /export.csv");
    }
    server.sendHeader("Content-Disposition", "attachment;
filename=ee4552_history.csv");
    server.send(200, "text/csv", csvHistory());
}

static void handleCommand()
{
    if (Config::EnableGatewayHttpDebug)
    {
        Serial.println("HTTP GET /command");
    }
    const uint8_t nodeId = uint8_t(server.arg("node").toInt());
    const CommandType command = commandFromText(server.arg("cmd"));
    int parameter = parseCommandParameter(command, server.arg("param"));

    if (nodeId == 0 || command == CommandType::None)
    {
        server.send(400, "text/plain", "invalid node/cmd");
        return;
    }

    if (command == CommandType::StartOta && parameter == 0)
    {
        parameter = int(millis() % 100000UL) + 1;
    }
    queueGatewayCommand(nodeId, command, parameter);

    Serial.printf("Queued HTTP command node=%u cmd=%s param=%d\n",
        nodeId, commandToText(command).c_str(), parameter);

    server.send(200, "text/plain", "queued");
}

static void handleSelfOta()
{
    if (Config::EnableGatewayHttpDebug)
    {
        Serial.println("HTTP GET /self_ota");
    }
    if (WiFi.status() != WL_CONNECTED)
    {
        server.send(503, "text/plain", "wifi not connected");
        return;
    }

    const String url = server.arg("url");
    const String md5 = server.arg("md5");
    if (!url.startsWith("http://") && !url.startsWith("https://"))
    {
        server.send(400, "text/plain", "missing url");
        return;
    }
}
```

```

server.send(200, "text/plain", "OTA started; gateway will reboot on success");
delay(250);

WiFiClient client;
HTTPClient http;
http.setTimeout(4000);
http.begin(client, url);
const int status = http.GET();
if (status != HTTP_CODE_OK)
{
    Serial.printf("OTA_FAILED http_status=%d\n", status);
    http.end();
    return;
}

const int contentLength = http.getSize();
if (contentLength <= 0 || !Update.begin(contentLength))
{
    Serial.printf("OTA_FAILED begin_error=%s size=%d\n", Update.errorString(),
contentLength);
    http.end();
    return;
}
if (md5.length() > 0)
{
    Update.setMD5(md5.c_str());
}

const size_t written = Update.writeStream(http.getStream());
if (written != size_t(contentLength))
{
    Serial.printf("OTA_FAILED written=%u expected=%d\n", unsigned(written),
contentLength);
    Update.abort();
    http.end();
    return;
}
if (!Update.end(true))
{
    Serial.printf("OTA_FAILED end_error=%s\n", Update.errorString());
    http.end();
    return;
}

Serial.printf("OTA_SUCCESS bytes=%u md5=%s\n", unsigned(written), md5.length()
> 0 ? md5.c_str() : "not_provided");
http.end();
delay(250);
ESP.restart();
}

static void handlePing()
{

```

```
    if (Config::EnableGatewayHttpDebug)
    {
        Serial.println("HTTP GET /ping");
    }
    server.send(200, "text/plain", "pong");
}

static void handleNotFound()
{
    if (Config::EnableGatewayHttpDebug)
    {
        Serial.print("HTTP 404 ");
        Serial.println(server.uri());
    }
    server.send(404, "text/plain", "not found");
}

void setupWebServer()
{
    server.on("/", handleRoot);
    server.on("/ping", handlePing);
    server.on("/export.csv", handleExport);
    server.on("/command", handleCommand);
    server.on("/self_ota", handleSelfOta);
    server.onNotFound(handleNotFound);
    server.begin();
    Serial.println("HTTP server listening on port 80");
    debugServer.begin();
    Serial.println("Debug HTTP server listening on port 8080");
}

static void webServerTask(void *)
{
    while (true)
    {
        handleDebugServer();
        server.handleClient();
        delay(2);
    }
}

void startWebServerTask()
{
    if (webTaskHandle != nullptr)
    {
        return;
    }

    xTaskCreatePinnedToCore(
        webServerTask,
        "web_server",
        8192,
        nullptr,
        1,
```

```

        &webTaskHandle,
        0);
    }

    static void handleDebugServer()
    {
        WiFiClient client = debugServer.available();
        if (!client)
        {
            return;
        }

        const uint32_t start = millis();
        while (client.connected() && !client.available() && millis() - start < 1000)
        {
            delay(1);
        }

        String requestLine;
        if (client.available())
        {
            requestLine = client.readStringUntil('\n');
            requestLine.trim();
        }

        if (Config::EnableGatewayHttpDebug)
        {
            Serial.print("DEBUG HTTP request: ");
            Serial.println(requestLine.length() > 0 ? requestLine : "(empty)");
        }

        client.print("HTTP/1.1 200 OK\r\n");
        client.print("Content-Type: text/plain\r\n");
        client.print("Connection: close\r\n");
        client.print("Content-Length: 4\r\n\r\n");
        client.print("pong");
        client.stop();
    }
}

```

src/gateway_main.cpp

```

#include <Arduino.h>
#include <WiFi.h>

#include "gateway_lora.h"
#include "gateway_state.h"
#include "gateway_web.h"
#include "project_config.h"

void setup()

```

```
{
  Serial.begin(Config::SerialBaud);
  delay(1000);
  Serial.println("==== EE4552 GATEWAY FIRMWARE ===");
  Serial.print("Firmware: ");
  Serial.println(Config::GatewayFirmwareVersion);

  loraReady = initLoRa();
  if (!loraReady)
  {
    Serial.println("LoRa init failed; WiFi/web will continue");
  }
  else
  {
    Serial.println("LoRa init OK");
  }

  connectWiFi();
  setupWebServer();
  startWebServerTask();
  Serial.println("Gateway ready");
}

void loop()
{
  if (otaReadyWaitActive && int32_t(millis() - otaReadyWaitDeadlineMs) > 0)
  {
    otaReadyWaitActive = false;
    Serial.println("OTA session failed: node did not report OTA_READY");
  }

  if (Config::EnableGatewayHeartbeatLog && millis() - lastHeartbeatMs >= 2000)
  {
    lastHeartbeatMs = millis();
    Serial.printf("Loop alive wifi=%d ip=%s\n", int(WiFi.status()),
WiFi.localIP().toString().c_str());
  }

  processLoRa();
  processSerialCommand();
  delay(1);
}
```